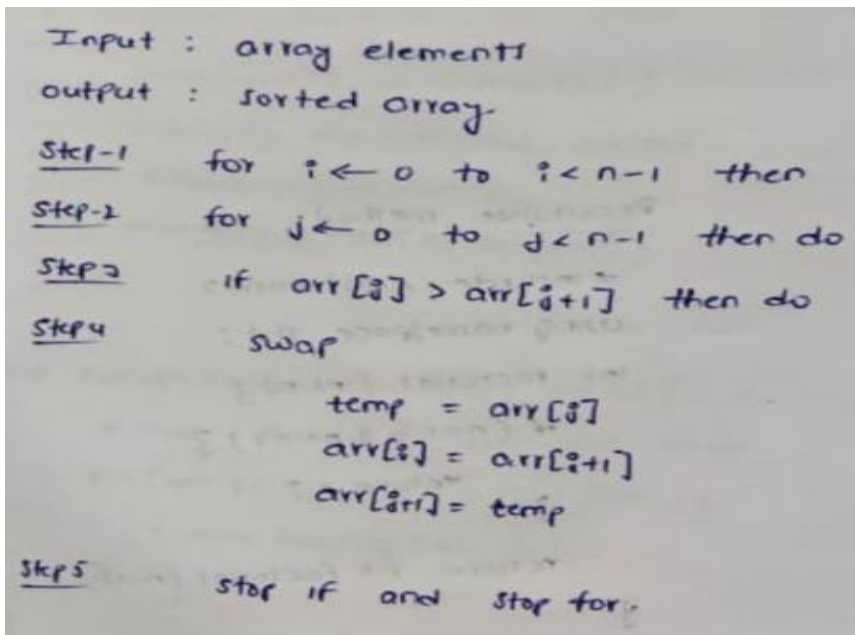


PRACTICAL-1

AIM: Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

BUBBLE SORT

ALGORITHM:



Input : array elements
output : sorted array.

Step-1 for $i \leftarrow 0$ to $i < n-1$ then

Step-2 for $j \leftarrow 0$ to $j < n-1$ then do

Step-3 if $arr[i] > arr[i+1]$ then do

Step-4 swap

$temp = arr[i]$
 $arr[i] = arr[i+1]$
 $arr[i+1] = temp$

Step-5 stop if and stop for.

CODE:

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    for (int i = 0; i < n - 1; i++) {
```

```

for (int j = 0; j < n - i - 1; j++) {
    if (arr[j] > arr[j + 1]) {
        int temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
    }
}
cout << "Sorted array: ";

for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
return 0;
}

```

OUTPUT:

```

Output

Enter the number of elements: 6
Enter 6 elements: 2 3 4 1 6 5
Sorted array: 1 2 3 4 5 6

=== Code Execution Successful ===

```

TIME COMPLEXITY:

Time complexity :- Both Worst and average case

```

for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - 1; j++) {
        if (arr[i] > arr[j + 1]) {
            temp = arr[i];
            arr[i] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}

```

$n-1$
 $n-1$
 1
 1
 1
 1

$T = (n-1)(n-1) \times 1 \times 1 \times 1 \times 1$
 $T = (n-1)^2$
 $T = n^2 - 2n + 1$
 $T = n^2 - n$
 $\boxed{T(n) = O(n^2)}$

BEST CASE:

Best case :-
When the array is already sorted
→ The array will be sorted for first pass only
 $T(n)$ for one pass $n-1$
 $T(n) = n-1$
 $T(n) = O(n)$

SELECTION SORT

ALGORITHM:

input : array elements
output : sorted array
step 1 - for $i \leftarrow 0$ to $i \leftarrow n-1$ then
step 2 - select $\min \leftarrow i$ then do
step 3 - for $j \leftarrow i+1$ to $j \leftarrow n$ then
step 4 - if $a[j] < a[\min]$ then do
step 5 - $\min \leftarrow j$
step 6 - stop if and for
step 7 - if $\min \neq i$ then do
step 8 - swap
 $\text{temp} = a[i]$
 $a[i] = a[\min]$
 $a[\min] = \text{temp}$
step 9 - stop if and for

CODE:

```
#include <iostream>
using namespace std;

class SelectionSort { public:
int size, arr[100], i, j, temp;

    void arrayinput()
    {        cout << "Enter array
size: ";
        cin >> size;

        cout << "Enter array elements: ";
        for (i = 0; i < size; i++)
        {            cin
>> arr[i];
        }
    }
    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
    void selectionsortlogic()
    {
        for (i = 0; i < size - 1; i++)
        {            int
min = i;

            for (j = i + 1; j < size; j++)
            {
                if (arr[j] < arr[min])
                {
min = j;
                }
            }

            if (min != i)
            {                temp = arr[i];
```

```
arr[i] = arr[min];
arr[min] = temp;
    }
}
};

int main()
{
    SelectionSort ss;

    ss.arrayinput();

    cout << "Before sorting: ";
    ss.printarray();

    ss.selectionsortlogic();

    cout << "After sorting: ";
    ss.printarray();

    return 0;
}
```

OUTPUT:

Output

```
Enter the number of elements: 6
Enter 6 elements: 2 3 4 1 6 5
Sorted array: 1 2 3 4 5 6

=== Code Execution Successful ===
```

TIME COMPLEXITY:

```

for (i=0; i<n-1; i++)      → n-1
{
    int min = i;           → 1
    for (j=i+1; j<n; j++)   → n
    {
        if (arr[j] < arr[min]) → 1
        {
            min = j;        → 1
        }
    }
    if (min != i)           → 1
    {
        int temp = arr[i];  → 1
        arr[i] = arr[min];  → 1
        arr[min] = temp;    → 1
    }
}

```

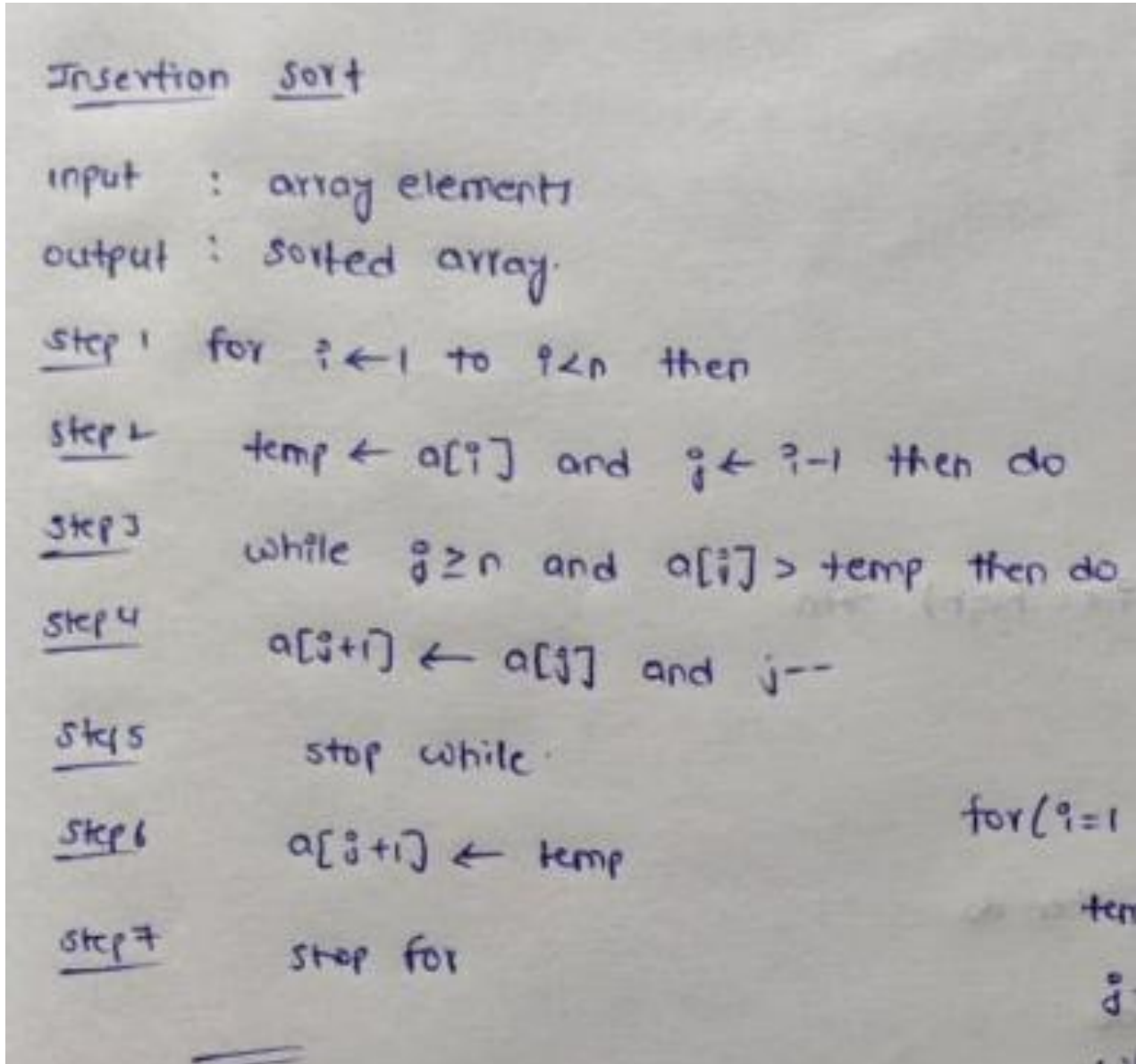
$$\begin{aligned}
 T &= (n-1)(n-1)(1 \times 1 \times 1 \times 1 \times 1 \times 1 \times 1) \\
 &= (n-1)^2 \\
 &= n^2 + 1 - 2n \\
 &= n^2
 \end{aligned}$$

$$T(n) = O(n^2)$$

Best = Average = Worst = $O(n^2)$

INSERTION SORT

ALGORITHM:



Insertion sort

input : array elements
output : sorted array.

step 1 for $i \leftarrow 1$ to $i \leq n$ then

step 2 temp $\leftarrow a[i]$ and $j \leftarrow i-1$ then do

step 3 while $j \geq 0$ and $a[j] > \text{temp}$ then do

step 4 $a[j+1] \leftarrow a[j]$ and $j--$

step 5 stop while

step 6 $a[j+1] \leftarrow \text{temp}$

step 7 stop for

CODE:

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

OUTPUT:

Output

```
Enter the number of elements: 4
Enter 4 elements: 3 4 2 6
Sorted array: 2 3 4 6
```

```
=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time complexity :- (Worst & Avg case)

```

for(i=1 ; i<n ; i++) {
    temp = a[i];
    j = i-1;
    while (j>0 && a[j]>temp) {
        a[j+1] = a[j];
        j--;
    }
    a[j+1] = temp;
}

```

→ n
→ 1
→ 1
→ n
→ 1
→ 1
→ 1

$T = n \times n$
 $T(n) = n^2$
 $T(n) = O(n^2)$

Best case

No of comparisons for

2nd element = 1
 3rd element = 1
 ⋮
 nth element = 1

Because the array will be sorted.

$T(n) = n-1$
 $T(n) = O(n)$

MERGE SORT-

ALGORITHM:

Merge Sort algorithm

input : array elements
output : sorted array

Step 1 MERGE (arr, low, mid, high) then

Step 2 set $i = \text{low}$, $j = \text{mid} + 1$, $k = \text{low}$.

Step 3 while $i \leq \text{mid}$ and $j \leq \text{high}$ then

Step 4 if $\text{arr}[i] \leq \text{arr}[j]$, then
 $\text{temp}[k] = \text{arr}[i]$
 $i = i + 1$
else
 $\text{temp}[k] = \text{arr}[j]$
 $j = j + 1$
END if
 $k = k + 1$
END while

Step 5 while $j \leq \text{high}$ then
 $\text{temp}[k] = \text{arr}[j]$
 $j = j + 1$
 $k = k + 1$
End while

Step 6 while $i \leq \text{mid}$ then
 $\text{temp}[k] = \text{arr}[i]$
 $i = i + 1$
 $k = k + 1$
End while

Step 7 for $i \leftarrow \text{low}$, $k \leftarrow 0$ to $i \leq \text{high}$ then
 $\text{arr}[i] = \text{temp}[k]$
stop for

Step 8 MERGE_SORT (arr, low, high) then do

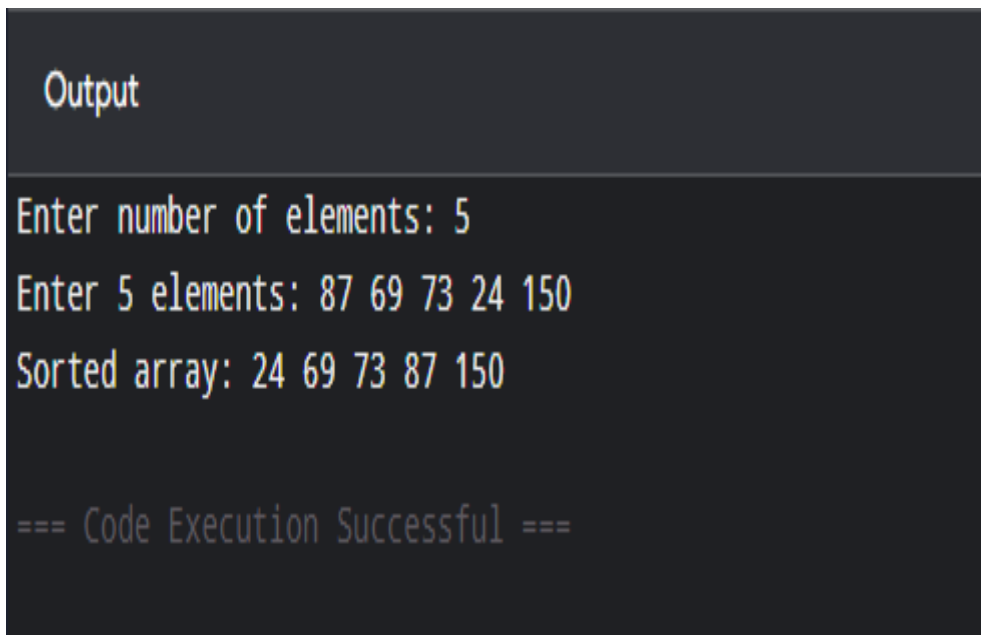
Step 9 if $\text{low} < \text{high}$ then do
 $\text{mid} = (\text{low} + \text{high}) / 2$
 MERGE_SORT (arr, low, mid)
 MERGE_SORT (arr, mid + 1, high)
 MERGE (arr, low, mid, high)
End if

CODE:

```
#include <iostream>
using namespace std;
void merge(int arr[], int low, int mid, int high) {
    int i = low;
    int j = mid + 1;
    int k = 0;
    int temp[high - low + 1];
    while (i <= mid && j <= high) {
        if (arr[i] < arr[j]) {
            temp[k] = arr[i];
            i++;
        }
        else {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }
    while (i <= mid) {
        temp[k] = arr[i];
        i++;
        k++;
    }
    while (j <= high) {
        temp[k] = arr[j];
        j++;
        k++;
    }
    for (int i = low, k = 0; i <= high; i++, k++) {
        arr[i] = temp[k];
    }
}
void mergeSort(int arr[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}
int main() {
    int n;
```

```
cout << "Enter number of elements: ";
cin >> n;
int arr[n];
cout << "Enter " << n << " elements: ";
for (int i = 0; i < n; i++) {
    cin >> arr[i];
}
mergeSort(arr, 0, n - 1);
cout << "Sorted array: ";
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
return 0;
}
```

OUTPUT:



Output

```
Enter number of elements: 5
Enter 5 elements: 87 69 73 24 150
Sorted array: 24 69 73 87 150

=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time complexity :- (Best, Average & Worst)

mergesort(arr, low, mid) ; $\rightarrow n/2$

mergesort(arr, mid+1, high) ; $\rightarrow n/2$

merge(arr, low, mid, high) ; $\rightarrow n$

$$T(n) = T(n/2) + T(n/2) + n$$

$$T(n) = 2T(n/2) + n$$

Master method

$$T(n) = aT(n/b) + f(n)$$

$$a=2 \quad b=2 \quad f(n)=n, \quad P=0, \quad K=1$$

$$n^{\log_b a} = n^{\log_2 2} = n$$

$$f(n) = n$$

$$n^{\log_2 2} = n$$

$$\text{calc 2 : } P > -1$$

$$T(n) = O(n \log^{P+1} n)$$

$$\boxed{T(n) = O(n \log n)}$$

QUICK SORT

ALGORITHM:

```
input :- array elements
output :- Sorted array

Step-1 :- Partition (arr, low, high) then
Step 2 :- Set pivot = arr[high]
           Set i = low - 1
Step 3 :- for j ← low to high - 1 then
Step 4       if arr[j] < pivot, then
                i = i + 1
                swap arr[i] and arr[j]
           End if
       End for
Step 5       swap arr[i + 1] and arr[high]
Step 6       Return i + 1
Step 7       QuickSort (arr, low, high) then
Step 8       if low < high then do
                PivotIndex = Partition (arr, low, high)
                QuickSort (arr, low, PivotIndex - 1)
                QuickSort (arr, PivotIndex + 1, high)
           End if
```

CODE:

```
#include <iostream>
using namespace std;
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {

        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    quickSort(arr, 0, n - 1);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

OUTPUT:

```

Output

Enter number of elements: 5
Enter 5 elements: 87 69 73 24 150
Sorted array: 24 69 73 87 150

=== Code Execution Successful ===

```

TIME COMPLEXITY:

Time complexity — (Best & Average case)

glucksort (arr, low, pp-1) : $\rightarrow n/2$
 gluck sort (arr, pp+1, high) : $\rightarrow n/2$
 partition (arr, low, high) : $\rightarrow n$

$$T(n) = T(n/2) + T(n/2) + n$$

$$T(n) = 2T(n/2) + n$$

Master method

$$T(n) = aT(n/b) + f(n)$$

$a = 2$ $b = 2$ $f(n) = n$, $P = 0$, $k = 1$

$$n \log_b a = n \log_2 2 = n$$

$$\log_b a = k$$

Case-2 $P > -1$

$$T(n) = \Theta(n \log^{k+1} n)$$

$$T(n) = \Theta(n \log n)$$

Worst case

n
 $\downarrow$
 $n-1$
 $\downarrow$
 $n-2$
 $\downarrow$
 $n-3$
 $\downarrow$
 $n-4$

$$T(n) = T(n-1) + n$$

$$T(n) = \frac{n(n+1)}{2}$$

$$T(n) = \frac{n^2 + n}{2}$$

$$T(n) = O(n^2)$$

CONCLUSION:

The practical successfully implemented and analyzed five important sorting algorithms: **Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort**. By comparing their working principles and time complexities, it was observed that **Bubble Sort, Selection Sort, and Insertion Sort** generally take $O(n^2)$ time in the average and worst cases, while **Merge Sort** provides $O(n \log n)$ performance consistently. **Quick Sort** also has an average time complexity of $O(n \log n)$, although its worst case can be $O(n^2)$ depending on the choice of pivot.

Thus, the practical helped in understanding how different sorting techniques work, how their performance varies with input size, and how **time complexity is useful for selecting an efficient algorithm** for a particular problem.